WSR-YOLO for PCB Defect Image Processing with Wavelet-Conditioned Sparse Routing

Jiefeng Liang[a], Lihua Luo[ab]*, Sijin Tan[a], Yanni Lin[a], Zhizhuo Zhao[a], and Zhaofeng Cai[a]
[a]Department of Computing, Guangdong University of Science and Technology, Dongguan 523083, China
[b]Faculty of Computer and Mathematical Sciences, Universiti Teknologi MARA, 40450 Shah Alam, Selangor, Malaysia
*Corresponding author: Lihua Luo, 260423630@qq.com
Project repository: https://github.com/master-abc/WSR-YOLO

***Abstract*—Traditional image processing for industrial visual inspection relies on hand-crafted filtering, thresholding, and edge operators, whereas WSR-YOLO is a modern deep-learning approach. WSR ranks P3 features with fixed Haar cues and refines an exact top-k budget. On DsPCBSD+, seven paired runs give 46.69±0.72 AP50:95 versus 46.54±0.36 for YOLO11s (p=0.8125); three descriptive DeepPCB pairs gain 4.20 points but increase clean-template alarms. At a 25% budget, routes enrich box cells by 3.16×/3.55×, although spatial priors and matched convolution explain much of the gain. WSR adds 13.3K parameters and 0.038 GFLOPs but is 1.18–1.21× slower. Negative-aware training reduces DeepPCB board-FPR from 53.5% to 27.5%; an exploratory paired-reference detector reaches 1.4% and 0.959 F1 under a changed input contract.**

***Keywords*—image processing, PCB defect detection, sparse routing, wavelets, false alarms**

I. INTRODUCTION

Automated optical inspection must detect small, low-contrast defects among repeated traces without alarming on normal boards. Positive-only benchmarks omit that operational risk, while route maps and FLOPs alone cannot establish accuracy, speed, or reliability.

WSR is therefore tested as an auditable probe. Fixed Haar responses rank P3 locations, and gather–refine–scatter transforms only k=round(ρHW) tokens. Paired seeds, same-budget and matched-capacity controls, device timing, and defect-free inputs test route enrichment, held-out accuracy, realized latency, and cue specificity.

Our exact-budget probe separates arithmetic savings from measured speed and exposes normal-board false alarms. Because the primary gain is nonsignificant and no variant passes the joint accuracy–latency gate, we report the evidence boundary rather than claim superiority.

II. RELATED WORK

Detectors span region proposals [1], set prediction [2], sparse proposals [3], and compact YOLO/DETR models [4], [5]. PCB protocols also differ: DeepPCB provides target/template pairs [6], DsPCBSD+ has nine AOI classes [7], and PCB-IND adds illumination and long-tail variation [8]. Results from PCB-YOLO and YOLO-HMC therefore remain protocol-dependent [9], [10].

Wavelet anti-aliasing and frequency attention [11], [12] motivate WSR, while dynamic token methods allocate input-dependent computation [13]. Yet scoring, indexing, and memory movement may dominate skipped arithmetic, requiring direct timing.

III. WAVELET-CONDITIONED SPARSE ROUTING

Let X∈R^(C×H×W) be the P3/8 feature, split into wave and refinement tensors Xw and Xr. Fixed grouped Haar convolution gives

$$(\boldsymbol{LL}, \boldsymbol{LH}, \boldsymbol{HL}, \boldsymbol{HH}) = \mathcal{H}(\boldsymbol{X}_w), \tag{1}$$

Normalized detail energy Ehf, low-frequency magnitude Ell, and local residual Gll enter a small router:

$$\boldsymbol{S} = R_\theta([\bar{E}_{hf}; \bar{E}_{ll}; \bar{G}_{ll}]) + \log(1 + \bar{E}_{hf} + \bar{G}_{ll}). \tag{2}$$

Flattening spatial locations, the router selects an exact budget

$$\mathcal{J} = \mathrm{TopK}(\boldsymbol{S}, k), \quad k = \min(HW, \max(4, \mathrm{round}(\rho HW))). \tag{3}$$

A normalized 3×3 kernel contextualizes Xr; only selected indices are gathered. For token tensor T,

$$\boldsymbol{T}'_{\mathcal{J}} = \boldsymbol{T}_{\mathcal{J}} + \sigma(\boldsymbol{S}_{\mathcal{J}}) \odot \mathrm{MLP}\left(\mathrm{LN}(\boldsymbol{T}_{\mathcal{J}} + \gamma(\tilde{\boldsymbol{T}}_{\mathcal{J}} - \boldsymbol{T}_{\mathcal{J}}))\right). \tag{4}$$

Unrouted tokens remain unchanged. A softmax gate fuses both branches with an identity skip. One block is inserted at P3 with ρ=0.25; confidence factors and detector losses train the router although hard indices receive no gradient (Figure 1).

IV. EXPERIMENTS

A. Protocol

Table 1 gives the frozen splits. DsPCBSD+ uses seven paired seeds; DeepPCB uses three descriptive pairs and audits 500 clean templates separately. Models share the YOLO11s checkpoint, 640-pixel input, 80-epoch limit, deterministic SGD, and augmentation. A pinned COCO evaluator [14] reports AP; RTX 3090 timing uses synchronized ABBA/BAAB cycles.

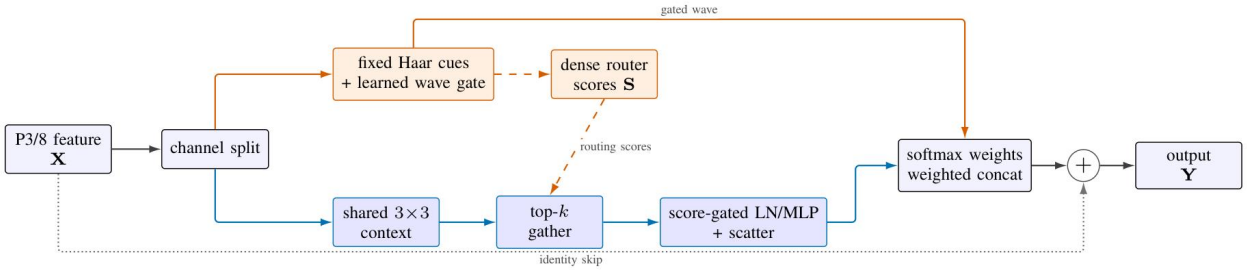

Figure 1. Compact WSR flow. Fixed Haar cues drive wave gating and dense routing; only the gathered channel MLP uses the exact ρHW budget.

Pretraining transfer must exceed 99%; paths, labels, boxes, hashes, and similarity are audited before training. P3/25% is selected on validation data before locked-test access; later studies are explicitly diagnostic or exploratory.

Table 1. Frozen dataset partitions after conversion and validation.

Dataset	Split	Images	Boxes
DsPCBSD+	Train	7,387	14,590
	Validation	821	1,594
	Locked test	2,051	4,092
DeepPCB	Train	850	5,870
	Validation	150	1,003
	Official test	500	3,140

Route recall is $|Mi \cap Bi|/|Bi|$; enrichment divides it by the selected fraction $|Mi|/Ni$. Board-FPR is the share of clean templates with a retained detection.

B. Accuracy

Table 2 reports architecture-controlled tests. DsPCBSD+ gains only 0.155 AP across four of seven wins (p=0.8125; 95% interval [−0.535, 0.844]). DeepPCB gains 4.196 points, but n=3 and p=0.25 preclude confirmation.

Table 2. Architecture-controlled test AP (%), mean±sample SD.

Dataset	Model	n	AP50:95	AP50
DsPCBSD+	YOLO11s	7	46.54±0.36	80.25±0.48
	WSR-YOLO11s	7	46.69±0.72	80.46±0.86
DeepPCB	YOLO11s	3	64.61±1.91	94.34±1.07
	WSR-YOLO11s	3	68.81±4.12	95.46±0.43

Figure 2 shows unstable DsPCBSD+ seed effects (−0.99 to +0.98; two classes decline), whereas all DeepPCB classes gain but reuse only three checkpoints. Figure 3 gives model-blind examples. Larger published detectors gain 2.75–6.39 AP with 1.75–3.55× the parameters, under different recipes.

Scale-specific changes are also inconsistent: the effect is neither seed-stable on the primary dataset nor uniform across object sizes. Class and scale slices remain diagnostic decompositions, not independent samples.

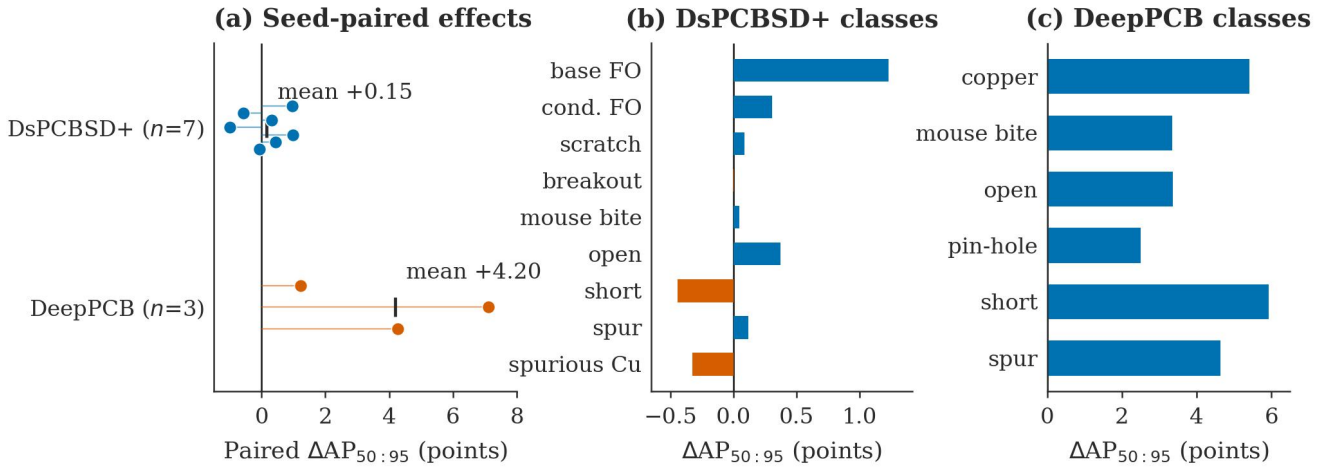

Figure 2. Paired and classwise accuracy differences. Dots are paired seeds; bars show per-class mean changes, with orange indicating regressions.

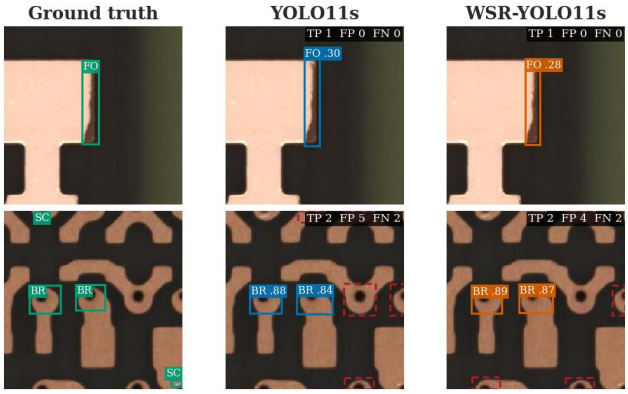

Figure 3. Model-blind DsPCBSD+ examples. Green: ground truth; blue/orange: true positives; dashed red: false positives. TP/FP/FN use class-matched IoU ≥0.5.

C. Routing, Controls, and Cost

At 25%, routes recall 79.11%/88.68% of box cells on DsPCBSD+/DeepPCB (3.164×/3.547× enrichment). Figure 4 shows that spatial priors explain much of the primary enrichment; cross-image shuffling confirms image dependence but not wavelet specificity. Equal fusion, no-HF, and matched convolution reach 48.314, 48.296, and 48.057 AP versus 48.025 for WSR, leaving capacity and optimization as plausible explanations.

The validation gate admitted WSR at +1.283 AP and 2.469× enrichment, but locked timing reversed the preliminary speed advantage.

Across 27 validation jobs, WSR reaches 48.025 AP versus 46.995 for the repeated baseline, with a 95% interval of [−0.158, 2.219]. The matched controls therefore do not isolate a unique wavelet mechanism.

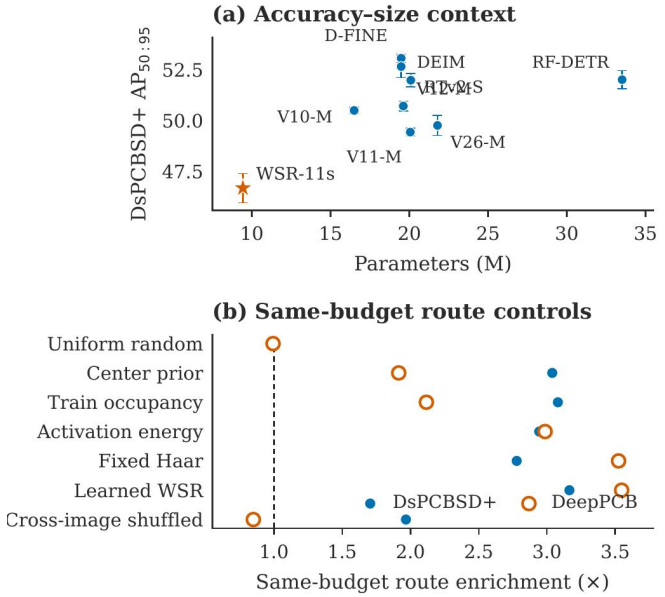

Figure 4. Model-size and route context. (a) DsPCBSD+ AP versus parameters. (b) Same-budget route enrichment; the dashed line is uniform expectation.

WSR adds 13,342 parameters (0.14%) and 0.038 GFLOPs (0.18%), yet latency is 1.184–1.206× baseline; every paired cycle interval excludes one. Dense support and unfused indexing therefore yield sparse arithmetic, not acceleration (Table 3).

The 95% ratio intervals are [1.166, 1.203]/[1.162, 1.227] on DsPCBSD+ FP32/FP16 and [1.193, 1.220]/[1.166, 1.204] on DeepPCB, confirming slowdown in all four settings.

Table 3. Compact validation and latency audit. Ratios are WSR/baseline.

Study	Variant/setting	ΔAP	Ratio
Control	WSR P3/25%	+1.030	—
	no HF cue	+1.301	—
	equal fusion	+1.319	—
	matched convolution	+1.062	—
Latency	DsPCBSD+ FP32/FP16	—	1.184/1.194
	DeepPCB FP32/FP16	—	1.206/1.185

D. Operational False Alarms

DeepPCB's 500 clean templates expose risk hidden by positive-only AP. At confidence 0.25, seed-13 WSR alarms on 56.6% of boards versus 48.0% for YOLO11s (Table 4). Negative-aware training lowers the three-seed board-FPR from 53.5% to 27.5% at 0.946 recall and raises AP from 68.81 to 72.44. The exploratory paired-reference policy reaches 1.4% board-FPR, AP 72.63, and 0.959 F1, but is post-hoc, single-seed, and changes the input contract (Figure 5).

Table 4. Seed-13 board-FPR (%) on 500 clean DeepPCB templates.

Model	0.05	0.10	0.25	0.50
YOLO11s	79.0	66.2	48.0	28.0
WSR-YOLO11s	87.6	77.0	56.6	35.0

Target-only mitigation preserves the sensor input; paired assistance requires a registered template. Validation-calibrated 1% caps transfer to 1.8%–6.2% test board-FPR with 0.790 mean recall, so production negatives remain necessary.

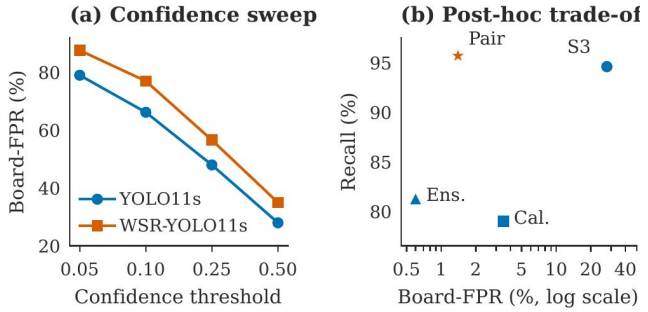

Figure 5. DeepPCB negative-template audit. (a) Target-only confidence sweep. (b) Recall–board-FPR trade-off; paired input changes the inference contract.

E. Auxiliary Robustness Evidence

Across eight corruptions and five severities, WSR averages 46.51 AP versus 42.68 and wins 24 of 40 conditions. Gains cluster in brightness, contrast, Gaussian noise, and JPEG, while three corruptions regress; clean-normalized retention is 65.6% versus 64.1%.

Low-/high-only reconstruction gives 67.43/0.95 AP for WSR and 66.22/1.67 for baseline. Removing LH, HL, or HH costs WSR 1.32, 0.70, or 1.68 points versus 1.59, 0.92, or 3.68 for baseline. Because the interventions modify detector input, they are mechanistic evidence rather than proof of internal Haar causality (Figure 6).

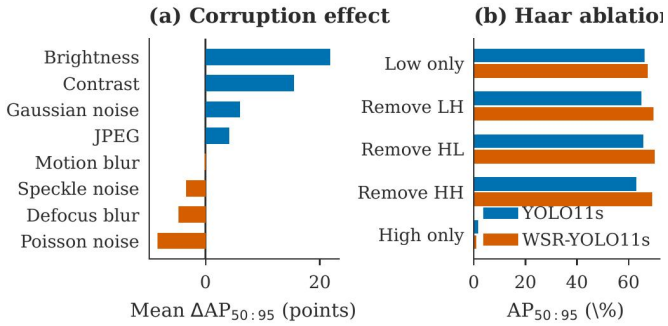

Figure 6. DeepPCB seed-13 diagnostics. (a) Mean WSR-minus-baseline AP under corruptions. (b) AP after matched input-Haar interventions.

V. FINDINGS AND LIMITATIONS

Route enrichment is not an accuracy surrogate, arithmetic sparsity is not systems efficiency, and positive-only AP hides normal-board alarms. A valid audit combines paired held-out accuracy, device timing, negative operating points, same-budget controls, matched capacity, and source-bound records. Seeds and latency cycles remain the sampling units; class, corruption, and threshold slices are decompositions rather than new samples.

A. Image-Processing Role and Deployment Gate

Filtering, binarization, and edge operators remain useful for normalization, registration, and ROI masks, but fixed thresholds are brittle under illumination and repeated copper patterns. WSR-YOLO instead learns multiscale context. Deployment freezes classwise thresholds under a validation board-FPR ceiling and reports locked-test AP, recall, board-FPR uncertainty, and end-to-end latency; camera or layout changes trigger recalibration, while loss of registration disables the paired branch.

Limits include three DeepPCB pairs, one GPU, incomplete comparator timing, hard top-k, and absent board/lot IDs. Reference assistance requires preregistration and independent production negatives. Runs bind Python 3.10, PyTorch 2.5.1/CUDA 12.1, Ultralytics 8.4.50, source revision, manifest, seed, checkpoint, and evaluator output; validation controls never read test labels.

VI. CONCLUSION

WSR concentrates selected P3 cells inside defects at little arithmetic cost, yet establishes neither a reliable primary accuracy gain nor acceleration, and target-only false alarms remain high. Its contribution is an auditable image-processing result and evaluation template. Future work should test fused sparse kernels, more paired seeds, group-safe splits, and independent reference-assisted evaluation.

ACKNOWLEDGEMENTS

The work was supported by the Teaching, Science, and Innovation Teaching and Learning Project of Guangdong University of Science and Technology under Grant (GKJXXZ2024008) and 2025 Undergraduate Innovation and Entrepreneurship Training Program Project under Grant (202513719002).

REFERENCES

[1] S. Ren et al., “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” NeurIPS, 2015. https://arxiv.org/abs/1506.01497

[2] N. Carion et al., “End-to-End Object Detection with Transformers,” ECCV, 2020. https://arxiv.org/abs/2005.12872

[3] P. Sun et al., “Sparse R-CNN: End-to-End Object Detection with Learnable Proposals,” CVPR, 2021. https://arxiv.org/abs/2011.12450

[4] G. Jocher and J. Qiu, “Ultralytics YOLO11,” version 11.0.0, 2024. https://github.com/ultralytics/ultralytics

[5] Y. Peng et al., “D-FINE: Redefine Regression Task in DETRs as Fine-grained Distribution Refinement,” ICLR, 2025. https://arxiv.org/abs/2410.13842

[6] S. Tang et al., “Online PCB Defect Detector on a New PCB Defect Dataset,” 2019. https://arxiv.org/abs/1902.06197

[7] S. Lv et al., “A Dataset for Deep Learning Based Detection of Printed Circuit Board Surface Defect,” Scientific Data, vol. 11, Art. no. 811, 2024. https://doi.org/10.1038/s41597-024-03656-8

[8] H. Yan et al., “Industrial Printed Circuit Board Surface Defect Dataset for Object Detection,” Scientific Data, 2026. https://doi.org/10.1038/s41597-026-07684-4

[9] J. Tang et al., “PCB-YOLO: An Improved Detection Algorithm of PCB Surface Defects Based on YOLOv5,” Sustainability, vol. 15, no. 7, Art. no. 5963, 2023. https://doi.org/10.3390/su15075963

[10] M. Yuan et al., “YOLO-HMC: An Improved Method for PCB Surface Defect Detection,” IEEE Trans. Instrum. Meas., vol. 73, pp. 1–11, Art. no. 2001611, 2024. https://doi.org/10.1109/TIM.2024.3351241

[11] Q. Li et al., “WaveCNet: Wavelet Integrated CNNs to Suppress Aliasing Effect for Noise-Robust Image Classification,” IEEE Trans. Image Process., vol. 30, pp. 7074–7089, 2021. https://doi.org/10.1109/TIP.2021.3101395

[12] Z. Qin et al., “FcaNet: Frequency Channel Attention Networks,” ICCV, 2021. https://arxiv.org/abs/2012.11879

[13] Y. Rao et al., “DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification,” NeurIPS, 2021. https://arxiv.org/abs/2106.02034

[14] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” ECCV, 2014. https://arxiv.org/abs/1405.0312